That is, the expected reward at play *k* for action *a* is the arithmetic mean of all the previous rewards we've received for taking action *a*. Thus, our previous actions and observations influence our future actions. We might even say some of our previous actions *reinforce* our current and future actions, but we'll come back to this later. The function $Q_k(a)$ is called a *value function* because it tells us the value of something. In particular, it is an *action-value function* because it tells us the value of taking a particular action. Since we typically denote this function with the symbol *Q*, it's also often called a *Q function*. We'll come back to value functions later and give a more sophisticated definition, but this will suffice for now.

2.2.1. Exploration and exploitation

When we first start playing, we need to play the game and observe the rewards we get for the various machines. We can call this strategy *exploration*, since we're essentially randomly exploring the results of our actions. This is in contrast to a different strategy we could employ called *exploitation*, which means that we use our current knowledge about which machine seems to produce the most rewards, and keep playing that machine. Our overall strategy needs to include some amount of exploitation (choosing the best lever based on what we know so far) and some amount of exploration (choosing random levers so we can learn more). The proper balance of exploitation and exploration will be important to maximizing our rewards.

How can we come up with an algorithm to figure out which slot machine has the largest average payout? Well, the simplest algorithm would be to just select the action associated with the highest Q value:

Table 2.3. Computing the best action, given the expected rewards

Math	Pseudocode

$\forall a_i \in A_k$	`def get_best_action(actions, history):`
	`  exp_rewards = [exp_reward(action, history)`
	`for action in`
	`actions]`